

PROJECT MS · CHARACTER FRAMEWORK

신규 캐릭터 개발 가이드

공통 BASE 안정화 개정 · 2026-08-17

캐릭터 작업자는 공통 API를 통해 네트워크 객체와 전투 기능을 사용합니다.

2026-08-17 BASE 변경사항

캐릭터별 구현 변경 없음

이번 개정은 캐릭터 스크립트의 개별 기능을 바꾸지 않고, 공통 BASE의 네트워크 경계와 확장 지점을 정리한 버전입니다.

1. 피해 처리와 네트워크 신뢰성

- 요청 피해량이 아니라 State Authority가 실제 적용한 피해량을 기준으로 콜백과 궁극기 게이지를 처리합니다.
- 공격자·팀·피해 출처·스킬 ID·피격 위치와 방향을 포함한 DamageRequest 정보를 끝까지 보존합니다.
- 자기 자신·아군 피해 정책과 RPC 송신자/소유권을 검증하고 NaN/Infinity 입력을 거부합니다.

2. 캐릭터 소유 객체 공통 API

- CharacterDeployable, CharacterSummon, CharacterThrowable을 공통 소유 객체로 관리합니다.
- 수명 정책: Manual / Health / Duration / HealthOrDuration.
- 최대 개수 초과 정책: RejectNew / DestroyOldest / DestroyNewest / Unlimited.
- 소유자 파괴 콜백과 소유 객체용 공통 피해·탐색 브리지를 제공합니다.

3. 물리 투척체와 퓨즈

- 퓨즈 시작 시점을 OnSpawn / OnGroundContact / Manual 중 선택하고 Fuse Seconds를 설정합니다.
- Dynamic Rigidbody2D, non-trigger Collider2D, NetworkRigidbody2D 구성을 사용합니다.
- OnGroundContact는 Ground 레이어가 필요하며, 만료 시 권한 측 콜백 후 공통 파괴 경로로 정리합니다.

4. 수명 주기·입력·검증기

- 소유자 이탈 시 Shared Master가 인계받아 Destroy 또는 ExpireNormally 정책을 수행합니다.
- 원격 이벤트·VFX 전달을 위해 파괴를 1틱 지연하고 렌더 측 파괴 콜백을 제공합니다.
- 입력 잠금 시 남은 입력을 제거하고 리셋 시 잠금·상태를 정리합니다.
- 캐릭터와 소유 객체·투척체 프리팹 구성을 검증기가 함께 점검합니다.

확장 가이드

지뢰·장판·터렛은 Deployable를 기반으로 능력 컴포넌트를 조합하고, 펫·분신은 Summon에서 시작합니다. 장기 저장이 필요한 소유권은 재사용될 수 있는 PlayerRef 대신 백엔드 사용자 식별자를 사용합니다. 경쟁 환경에서는 API 모양을 유지한 채 객체 권한을 Master/Server로 이전합니다.

후배 작업자 체크

SpawnOwnedEntity / SpawnThrowable 사용 · Runner.Spawn 직접 호출 금지 · 직접 Runner.Despawn 대신 공통 Destroy API 사용 · 수명/퓨즈/최대 개수/소유자 이탈 정책 설정 · Tools > Project MS > Character > Validate Selected Character Asset 실행 · 2인 Shared 테스트

현재 캐릭터 BASE 공통 API

2026-08-17 기준

캐릭터 작업자는 아래 공통 API를 우선 사용합니다. 세부 시그니처와 예제는 각 기능 장을 확인하세요.

구분	대표 API / 사용 목적
행동·잔탄·쿨다운	TryConsumeAction · SetAmmo · AddAmmo · GetAmmo · StartCooldown · IsCooldownReady
이동·상태·타이머	SetMoveSpeedMultiplier · ApplySlow · RemoveSlow · LockCharacterInput · UnlockCharacterInput · Schedule · CancelSchedule
방향	FacingDirection · AimDirection · IsFacingDirection · IsTargetBehind
피해·대상	DealDamage · FindDamageablesInCircle · ModifyOutgoingDamage · OnDamageDealt · OnDamageableDealt
발사체	SpawnProjectile · DespawnProjectile · OnProjectileDespawnd
캐릭터 소유 객체	SpawnOwnedEntity · DestroyOwnedEntity · GetOwnedEntities · DestroyOwnedEntities
물리 투척체	SpawnThrowable · StartThrowableFuse
수명 주기	OnCharacterSpawned · OnCharacterDespawnd · OnResetCharacter · OnOwnedEntityDestroyed

사용 원칙

호출: 캐릭터 스크립트에서 공통 메서드를 사용합니다.
오버라이드: Modify/On 계열 혹은 필요한 경우에만 확장하고 base 호출 규칙을 지킵니다.
자동 처리: 네트워크 권한, 등록 해제, 파괴 통지와 공통 리셋은 BASE가 담당합니다.
금지: 캐릭터 스크립트에서 Runner.Spawn을 직접 호출하지 않습니다. Character 전용 (Networked)/(Rpc)가 꼭 필요하다면 공통 API 확장 가능성을 먼저 검토하고 코드 리뷰를 거칩니다.

문서를 먼저 보는 방법

이 문서는 어려운 Unity 용어보다 실제 작업 순서를 먼저 설명한다. 코드에서 반드시 사용해야 하는 이름만 괄호 안에 함께 표시한다.

먼저 찾고 싶은 내용	이동할 장
전체 구조와 작업 흐름	1장
폴더별 책임과 수정 가능한 파일	2장
캐릭터 프리팹 구조와 실제 만드는 방법	3장
체력, 이동 속도, 데미지 입력 방법	4장
핑타, Q, E, 궁극기, 패시브 코드 작성	5장
공통 캐릭터 기능 사용법	6장
총알, 화살 같은 투사체 프리팹 설정	7장
신규 캐릭터 개발 순서와 네트워크 주의점	8장
마지막 점검 목록	9장

가장 중요한 원칙: 캐릭터 개발자는 캐릭터별 스크립트, 캐릭터 수치 파일, 캐릭터별 프리팹만 수정한다. 공통 부모 코드와 네트워크 코드는 직접 수정하지 않는다.

이 문서에서 사용하는 쉬운 용어

문서에서 먼저 부르는 이름	코드 또는 Unity 용어	뜻
공통 부모 코드	<code>CharacterBase</code>	입력, 네트워크, 체력, 쿨타임 같은 공통 처리를 담당한다.
캐릭터 수치 파일	<code>CharacterDefinition</code>	최대 체력, 이동 속도, 스킬 데미지와 쿨타임을 저장한다.
공통 움직임 설정	<code>CharacterVisualProfile</code>	점프 늘어남, 착지 놀림, 흔들림 같은 표현 수치를 저장한다.
공통 캐릭터 프리팹	<code>CharacterTemplate.prefab</code>	모든 캐릭터가 공유하는 기본 오브젝트 구조다.
캐릭터별 프리팹	Prefab Variant	공통 프리팹과 연결된 상태로 캐릭터별 차이만 저장한 프리팹이다.
위치 기준점	Socket, <code>Transform</code>	공격, 총알, 무기, 효과가 시작될 위치를 표시하는 빈 오브젝트다.
이번 행동 정보	<code>CharacterActionContext</code>	이번 공격의 방향, 위치, 데미지를 묶어서 전달한다.
투사체	Projectile	총알, 화살, 마법탄처럼 날아가 충돌하는 공격 오브젝트다.
시각 효과	VFX	총구 섬광, 폭발, 피격 불꽃처럼 화면에만 보이는 효과다.

1. 한눈에 보는 신규 캐릭터 제작 순서

신규 캐릭터는 아래 순서로 만든다. 공통 코드를 수정하지 않고 캐릭터별 스크립트, 수치 파일, 프리팹을 차례대로 완성한다.

순서	개발 작업	사용하는 파일 또는 기능	결과
1	템플릿 스크립트를 복사한다.	<code>CharacterTemplate.cs</code>	캐릭터별 스크립트 생성
2	체력, 이동, 데미지, 쿨타임을 입력한다.	<code>CharacterDefinition</code>	캐릭터 수치 파일 완성
3	공통 프리팹을 바탕으로 캐릭터별 프리팹을 만든다.	<code>CharacterTemplate.prefab</code>	기본 오브젝트 구성 완성
4	평타, Q, E, 궁극기를 하나씩 작성한다.	공통 캐릭터 API	캐릭터 전투 규칙 완성
5	Body, 공격 위치, 투사체와 효과를 연결한다.	Inspector	프리팹 연결 완성
6	자동 검사와 2인 테스트를 진행한다.	Validator, Fusion	구성과 동작 확인

템플릿 스크립트 복사
 ↓
 캐릭터 수치 파일 만들기
 ↓
 캐릭터별 프리팹 만들기
 ↓
 평타부터 스킬을 하나씩 구현
 ↓
 Body, 공격 위치, 투사체, 효과 연결
 ↓
 자동 검사와 Fusion 2인 테스트

누가 무엇을 수정하는가

영역	담당 내용	신규 캐릭터 개발자가 수정하는가?
<code>CharacterBase</code>	입력 실행, 체력, 쿨타임, 공통 상태 관리	아니요
<code>CharacterInputHandler</code>	키와 마우스 입력 수집	아니요
<code>CharacterMovementHandler</code>	이동, 점프, 대시, 지면 판정	보통 수정하지 않음
<code>CharacterHealth</code>	체력 감소, 회복, 사망	아니요
<code>CharacterCooldownHandler</code>	평타와 스킬 쿨타임	아니요
<code>CharacterVisualController</code>	점프, 착지, 피격, 조준 표현	프리팹에서 설정만 연결
캐릭터별 스크립트	평타, Q, E, 궁극기, 패시브 규칙	예
<code>CharacterDefinition</code>	체력, 이동, 데미지, 쿨타임 수치	예
캐릭터별 프리팹	스프라이트, 공격 위치, 투사체와 효과 연결	예

2. 폴더별 역할과 수정 범위

```

Character
├── Runtime
│   ├── Core
│   │   ├── CharacterBase.cs
│   │   ├── CharacterActionContext.cs
│   │   └── CharacterSockets.cs
│   ├── Data
│   │   ├── CharacterDefinition.cs
│   │   └── CharacterVisualProfile.cs
│   ├── Modules
│   │   ├── CharacterInputHandler.cs
│   │   ├── CharacterMovementHandler.cs
│   │   ├── CharacterHealth.cs
│   │   └── CharacterCooldownHandler.cs
│   ├── Combat
│   │   ├── CharacterCombatQuery2D.cs
│   │   └── CharacterProjectile.cs
│   ├── Visual
│   │   ├── CharacterVisualController.cs
│   │   └── Solver
│   └── Examples
│       └── CharacterTemplate.cs
├── Editor
│   └── CharacterPrefabValidator.cs
└── Docs
  
```

폴더	쉬운 설명	신규 캐릭터 작업 중 수정 여부
Core	모든 캐릭터가 공유하는 실행 순서와 API	수정하지 않음
Data	캐릭터 수치와 공통 움직임 설정	새 파일 생성 또는 값 설정
Modules	입력, 이동, 체력, 쿨타임 실제 구현	수정하지 않음
Combat	범위 검색과 공통 투사체	수정하지 않음
Visual	모든 캐릭터가 공유하는 화면 표현	프리팸에서 연결만 함
Examples	새 캐릭터 스크립트를 시작할 때 참고할 예제	복사해서 사용
Editor	프리팸 구성이 맞는지 검사하는 메뉴	수정하지 않음

신규 캐릭터 전용 폴더를 따로 만들고 그 안에 캐릭터 스크립트, 캐릭터 수치 파일, 캐릭터별 프리팸을 모아 두면 찾기 쉽다.

3. 캐릭터 프리팹 구조와 실제 만드는 방법

프리팹은 캐릭터 한 명을 구성하는 오브젝트와 컴포넌트를 파일처럼 저장한 것이다. 모든 캐릭터는 같은 기본 뼈대를 사용하고, 캐릭터별 프리팹에서 스프라이트와 스킬 연결만 바꾼다.

3.1 공통 캐릭터 프리팹 구조

```

CharacterRoot
├─ 캐릭터 스크립트
├─ Rigidbody2D
├─ Collider2D
├─ NetworkObject
├─ NetworkRigidbody2D
├─
├─ VisualRoot
│   ├── CharacterVisualController
│   ├── Body
│   ├── AttackOrigin
│   ├── ProjectileOrigin    필요한 캐릭터만
│   ├── WeaponRoot         필요한 캐릭터만
│   ├── EffectOrigin        필요한 캐릭터만
│   ├── WorldCanvas         필요한 캐릭터만
│   └─ DustEffect           필요한 캐릭터만

```

3.2 공통 프리팹은 담당자가 처음 한 번 만든다

공통 시스템 담당자가 Unity에서 다음 순서로 `CharacterTemplate.prefab`을 만든다.

- 1 Hierarchy에서 빈 오브젝트를 만들고 이름을 `CharacterTemplate`로 변경한다.
- 2 루트에 `NetworkObject`, `NetworkRigidbody2D`, `Rigidbody2D`, `Collider2D`를 추가한다.
- 3 `CharacterBase`는 직접 붙일 수 없으므로 임시 스크립트인 `CharacterTemplate.cs`를 추가한다.
- 4 `VisualRoot`, `Body`, 필요한 위치 기준점 오브젝트를 자식으로 만든다.
- 5 `VisualRoot`에 `CharacterVisualController`를 추가한다.
- 6 Hierarchy의 `CharacterTemplate`을 Project 창으로 끌어 `CharacterTemplate.prefab`으로 저장한다.

권장 저장 위치:

```
Assets/00.Main/02.Prefab/Character/Common/CharacterTemplate.prefab
```

공통 프리팹은 팀에서 하나만 만든다. 신규 캐릭터 개발자가 매번 처음부터 다시 만들지 않는다.

3.3 공통 구조를 이용해 캐릭터별 프리팹 만들기

Unity에서는 공통 프리팹과 연결을 유지한 복사본을 `Prefab Variant`라고 부른다. 문서에서는 이해하기 쉽게 캐릭터별 프리팹이라고 부른다.

- 1 Project 창에서 `CharacterTemplate.prefab`을 선택한다.
- 2 마우스 오른쪽 버튼을 누른다.
- 3 Unity 메뉴에서 `Create > Prefab Variant`를 선택한다.
- 4 생성된 프리팹 이름을 캐릭터 이름으로 바꾼다. 예: `NewCharacter.prefab`.

- 5 새 프리팹을 두 번 눌러 Prefab Mode로 연다.
- 6 임시 `CharacterTemplate` 컴포넌트를 제거한다.
- 7 새로 만든 캐릭터 스크립트를 추가한다.
- 8 캐릭터 수치 파일, 공통 비주얼, 위치 기준점을 연결한다.

```
CharacterTemplate.prefab
├── NewCharacter.prefab
├── SwordCharacter.prefab
└── GunCharacter.prefab
```

캐릭터별 프리팹에서 수정할 항목:

항목	무엇을 설정하는가
캐릭터 스크립트	해당 캐릭터의 평타와 스킬 코드
<code>CharacterDefinition</code>	체력, 이동, 데미지, 쿨타임
Body	스프라이트와 Animator
Collider2D	캐릭터 몸 크기에 맞춘 충돌 범위
위치 기준점	공격, 투사체, 무기, 효과가 시작될 위치
스킬 프리팹	총알, 화살, 폭발 등 필요한 오브젝트
Visual 설정	움직임과 피격에 반응할 오브젝트

주의할 점:

- 캐릭터별 변경사항은 해당 캐릭터 프리팹에만 저장한다.
- `CharacterTemplate`와 캐릭터 스크립트를 동시에 붙이지 않는다.
- `Overrides > Apply All to Base`를 누르면 캐릭터별 설정이 공통 프리팹에 들어갈 수 있으므로 누르지 않는다.

3.4 공격 위치 기준점 사용법

공격 위치 기준점은 캐릭터 프리팹 안에 빈 GameObject를 만들고, 그 `Transform`을 원하는 위치로 옮긴 뒤 `CharacterBase`의 Socket 설정에 연결해서 사용한다.

`CharacterSockets`는 프리팹에 따로 추가하는 컴포넌트가 아니다. 캐릭터 루트의 `CharacterBase` 컴포넌트 안에 있는 `Gameplay Sockets > Sockets` 설정 묶음이다.

용도	연결할 Transform의 위치	CharacterBase에 연결하는 칸	비워 두면
공격 판정	검이나 주먹의 공격 판정을 시작할 위치	<code>Attack Origin</code> / 코드: <code>AttackOrigin</code>	캐릭터 루트 위치 사용
투사체	총알, 화살, 마법탄이 생성될 위치	<code>Projectile Origin</code> / 코드: <code>ProjectileOrigin</code>	Attack Origin 사용
무기 표현	검, 총, 활처럼 화면에 보이는 무기를 붙일 위치	<code>Weapon Root</code> / 코드: <code>WeaponRoot</code>	연결된 위치 없음
공격 효과	섬광, 폭발, 마법진 같은 공격 효과가 나타날 위치	<code>Effect Origin</code> / 코드: <code>EffectOrigin</code>	Attack Origin 사용

사용 방법:

- 1 캐릭터별 프리팹을 열고 캐릭터를 따라 움직일 위치 아래에 빈 GameObject를 만든다. 보통 `VisualRoot` 아래에 `Sockets`라는 빈 부모를 만들고 그 안에 모아 둔다.

- 2 빈 GameObject의 이름을 `AttackOrigin`, `ProjectileOrigin`처럼 용도에 맞게 정한다.
- 3 Scene 창에서 빈 GameObject의 `Transform`을 실제 공격이나 투사체가 시작될 위치로 옮긴다.
- 4 캐릭터 프리팹의 루트 오브젝트를 선택한다.
- 5 Inspector의 `CharacterBase > Gameplay Sockets > Sockets`를 펼친다.
- 6 만든 GameObject를 `Attack Origin`, `Projectile Origin`, `Weapon Root`, `Effect Origin` 중 사용하는 칸에 끌어다 놓는다.
- 7 캐릭터 코드에서는 연결된 위치를 `AttackOrigin`, `ProjectileOrigin`, `WeaponRoot`, `EffectOrigin`으로 사용한다.

캐릭터 프리팹에 빈 GameObject 생성
 → Transform으로 위치 조정
 → 캐릭터 루트의 CharacterBase 선택
 → Gameplay Sockets > Sockets의 해당 칸에 연결
 → 캐릭터 코드에서 AttackOrigin 등의 공통 위치 사용

사용하지 않는 위치는 만들거나 연결하지 않아도 된다. 예를 들어 투사체를 사용하지 않는 근접 캐릭터라면 `Projectile Origin`을 비워 둘 수 있다.

4. 캐릭터 수치와 움직임 설정

4.1 캐릭터 수치 파일 만들기

Unity Project 창에서 다음 메뉴를 선택한다.

Create > Project MS > Character > Definition

CharacterDefinition에는 실행 중에 자주 바뀌지 않는 기본 수치를 저장한다.

설정 묶음	입력하는 내용
기본 정보	표시 이름, 최대 체력
이동	이동 속도, 가속도, 점프 힘, 지면 Layer
점프 보정	점프 입력 보관 시간, 코요테 타임
대시	대시 힘과 지속 시간
자동 점프	Auto Hop 사용 여부와 수치
공격	평타, Q, E, 궁극기 데미지와 쿨타임
입력	기본 키 설정

현재 체력, 남은 쿨타임, 일시적인 버퍼처럼 실행 중 변하는 값은 이 파일에 저장하지 않는다.

4.2 공통 움직임 설정 사용하기

팀에서 정한 기본 **CharacterVisualProfile**을 먼저 사용한다. 캐릭터의 무게감이나 탄성이 확실히 달라야 할 때만 별도 파일을 만든다.

Create > Project MS > Character > Visual Profile

설정할 수 있는 예:

- 점프할 때 세로로 늘어나는 정도
- 착지할 때 납작해지는 정도
- 가만히 있을 때 흔들리는 정도
- 피격 시 색상 변화
- 팔이나 장식이 흔들렸다 돌아오는 힘과 감쇠

움직임 설정은 게임 판정에 영향을 주지 않는다. 화면에 보이는 반응만 조절한다.

5. 평타, 스킬, 패시브 코드 작성

5.1 먼저 구분할 것

구분	쉬운 뜻	개발자의 작업
캐릭터가 구현	캐릭터마다 공격 방식이 달라 비어 있는 함수	필요한 함수에 코드 작성
필요할 때 구현	기본 동작이 있거나 빈 이벤트 함수	특성이 있을 때만 작성
호출해서 사용	공통 부모 코드에 실제 기능이 이미 구현됨	스킬 안에서 함수 호출
자동 처리	입력 실행, 쿨타임, 공통 상태 관리	작성하지 않음

캐릭터가 주로 구현하는 함수:

함수	언제 구현하는가	구현하지 않으면
<code>OnBasicAttack</code>	평타가 있을 때	평타 입력이 실행되지 않음
<code>OnSkillQ</code>	Q 스킬이 있을 때	Q 입력이 실행되지 않음
<code>OnSkillE</code>	E 스킬이 있을 때	E 입력이 실행되지 않음
<code>OnUltimate</code>	궁극기가 있을 때	궁극기 입력이 실행되지 않음
<code>OnDash</code>	기본 대시와 다른 특수 대시일 때	공통 기본 대시 사용
패시브 이벤트 함수	피격, 공격 성공, 사망 등에 반응할 때	추가 반응 없음

5.2 새 캐릭터 스크립트 시작하기

- 1 `CharacterTemplate.cs`를 복사한다.
- 2 파일 이름을 캐릭터 이름으로 바꾼다.
- 3 클래스 이름도 파일 이름과 같게 바꾼다.
- 4 사용하는 공격 함수만 작성한다.

```
public sealed class NewCharacter : CharacterBase
{
    protected override bool OnBasicAttack(CharacterActionContext context)
    {
        // 평타 규칙
        return true;
    }

    protected override bool OnSkillQ(CharacterActionContext context)
    {
        // Q 스킬 규칙
        return true;
    }

    protected override bool OnSkillE(CharacterActionContext context)
    {
        // E 스킬 규칙
        return true;
    }

    protected override bool OnUltimate(CharacterActionContext context)
    {
        // 궁극기 규칙
        return true;
    }
}
```

true는 공격이 정상 실행됐다는 뜻이다. 이때 공통 부모 코드가 쿨타임과 후속 이벤트를 처리한다. **false**는 프리패스 누락이나 조건 불충족으로 공격을 취소했다는 뜻이며 쿨타임이 시작되지 않는다.

5.3 공격 방식에 맞는 함수 고르기

무기마다 새로운 부모 클래스를 만들지 않는다. 공격 모양에 맞는 공통 함수를 선택한다.

만들려는 공격	사용하는 공통 함수
총알, 화살, 마법탄	<code>SpawnProjectile</code>
검이나 부채꼴 근접 공격	<code>FindEnemiesInArc + DealDamage</code>
자신 중심 원형 공격	<code>FindEnemiesInCircle + DealDamage</code>
직선 관통 공격	<code>FindEnemiesInLine + DealDamage</code>
사각형 범위 공격	<code>FindEnemiesInBox + DealDamage</code>
기본 대시	별도 구현 없이 공통 대시 사용
특수 돌진 또는 회피	<code>StartDash</code>
공격 시각 효과	<code>PlayActionEffect</code>

5.4 패시브 작성 위치

매 프레임 `Update()`를 새로 만들기보다 상황에 맞는 공통 이벤트 함수를 사용한다.

```
protected override void OnDamaged(CharacterDamageInfo damage)
{
    // 피해를 받았을 때
}

protected override void OnDamageDealt(CharacterBase target, float requestedDamage)
{
    // 공격이 적에게 적용됐을 때
}

protected override void OnPassiveTick(float deltaTime)
{
    // 반복해서 조건을 확인해야 하는 지속 패시브
}
```

6. 공통 캐릭터 기능 사용법

이 장의 함수는 캐릭터별 스크립트 안에서 호출한다. 각 함수가 언제 필요한지와 어떤 값을 전달하는지만 확인하면 된다.

`CharacterBase`가 이미 구현한 것: 아래 공통 함수의 실제 처리, 네트워크 상태, 투사체 생성과 종료 처리다. 캐릭터 코드에서는 그대로 호출한다. 캐릭터 하위 클래스가 작성할 것: `OnBasicAttack`, `OnSkillQ`, `OnSkillE`, `OnUltimate` 같은 공격 규칙과, 필요할 때만 쓰는 `ModifyOutgoingDamage`, `OnProjectileDespawned` 같은 이벤트다.

6.1 이번 행동 정보 읽기

평타와 스킬 함수가 받는 `context`에는 이번 행동에 필요한 값이 들어 있다.

값	쉬운 뜻	사용 예
<code>context.Action</code>	어떤 행동인지	평타, Q, E, 궁극기 구분
<code>context.Origin</code>	기본 공격 시작 위치	범위 공격 중심
<code>context.AimDirection</code>	조준 방향	투사체, 돌진 방향
<code>context.AimWorldPosition</code>	마우스가 가리키는 월드 위치	설치 지점
<code>context.Damage</code>	수치 파일에서 가져온 이번 공격 데미지	<code>DealDamage</code> 에 전달
<code>context.AimAngle</code>	조준 방향을 각도로 바꾼 값	효과 회전

캐릭터 스크립트에서 마우스 위치와 데미지를 다시 계산하지 않는다. `context` 값을 사용해야 네트워크 결과가 일정해진다.

6.2 범위 안의 적 찾기

함수	검색 모양	대표 사용 예
<code>FindEnemiesInCircle</code>	원	자신 중심 폭발, 오라
<code>FindEnemiesInBox</code>	회전 가능한 사각형	내려찍기, 넓은 베기
<code>FindEnemiesInLine</code>	폭이 있는 직선	빔, 관통 공격
<code>FindEnemiesInArc</code>	부채꼴	검, 주먹, 원뿔 공격

원형 공격 예:

```
foreach (CharacterBase target in FindEnemiesInCircle(
    transform.position,
    skillRadius,
    targetLayer))
{
    DealDamage(target, context.Damage);
}
```

검 공격 예:

```
foreach (CharacterBase target in FindEnemiesInArc(
    AttackOrigin.position,
    context.AimDirection,
    swordRange,
    swordAngle,
    targetLayer))
{
    DealDamage(target, context.Damage);
}
```

6.3 데미지와 회복

```
DealDamage(target, context.Damage);
Heal(healAmount);
```

`DealDamage`를 사용하면 공통 체력 규칙에 맞게 피해가 적용된다. 대상의 체력을 캐릭터 스크립트에서 직접 빼지 않는다.

`Heal`은 현재 캐릭터 자신의 체력을 회복한다. 최대 체력을 넘지 않도록 공통 체력 코드가 처리한다.

6.4 투사체 생성

```
SpawnProjectile(
    projectilePrefab,
    ProjectileOrigin.position,
    context.AimDirection,
    projectileSpeed,
    context.Damage,
    targetLayer);
```

전달값	뜻
<code>projectilePrefab</code>	생성할 총알, 화살, 마법탄 프리팹
<code>ProjectileOrigin.position</code>	투사체가 처음 나타날 위치
<code>context.AimDirection</code>	날아갈 방향
<code>projectileSpeed</code>	이동 속도
<code>context.Damage</code>	맞았을 때 적용할 데미지
<code>targetLayer</code>	공격 대상으로 인정할 Layer

캐릭터 스크립트에서 투사체 생성 기능을 따로 만들지 않는다. `CharacterBase.SpawnProjectile`을 사용하고, 프리팹을 만들고 연결하는 방법은 7장을 따른다. `CharacterProjectile.Initialize`을 캐릭터 코드에서 직접 호출하지 않는다. 예전 5개 인자 초기화 함수는 발사자 정보를 남길 수 없어 의도적으로 컴파일 오류가 나게 막아 두었다.

6.5 대시와 시각 효과

일반 대시는 `OnDash`를 작성하지 않아도 실행된다.

특수 돌진:

```
StartDash(context.AimDirection, dashPower, dashDuration);
```

행동 시각 효과:

```
PlayActionEffect(
    context.Action,
    EffectOrigin.position,
    context.AimAngle);
```

`PlayActionEffect`는 다른 플레이어 화면에도 같은 효과를 요청한다. 데미지와 충돌 판정에는 사용하지 않는다.

6.6 행동 사용, 잔탄, 쿨타임

행동은 `CharacterActionType.BasicAttack`, `SkillQ`, `SkillE`, `Dash`, `Ultimate`로 구분한다. 아래 함수는 `CharacterBase`가 이미 구현했으므로 스킬 안이나 필요한 이벤트에서 호출한다.

하고 싶은 일	사용하는 함수	쉬운 뜻
행동을 켜거나 끄기	<code>SetActionEnabled(action, enabled)</code>	<code>false</code> 면 그 행동은 사용할 수 없다.
켜져 있는지 보기	<code>IsActionEnabled(action)</code>	행동 잠금 조건을 만들 때 쓴다.
잔탄 정하기	<code>SetActionCharges(action, charges)</code>	-1은 무제한, 0은 사용할 수 없음이다.
잔탄 더하거나 빼기	<code>AddActionCharges(action, amount)</code>	재장전이나 보상에 쓴다.
남은 잔탄 보기	<code>GetActionCharges(action)</code>	UI나 다음 사용 조건에 쓴다.
기본 쿨타임 바꾸기/되돌리기	<code>SetCooldownDuration</code> , <code>ResetCooldownDuration</code>	이 캐릭터의 행동 시간만 바꾼다.
자동 쿨타임 켜기/끄기	<code>SetAutoCooldown(action, enabled)</code>	성공한 행동 뒤 자동 시작을 정한다.
쿨타임 시작/지우기/보기	<code>StartCooldown</code> , <code>ClearCooldown</code> , <code>GetCooldownRemaining</code>	수동 시작, 즉시 해제, 남은 시간 확인에 쓴다.

```
SetActionCharges(CharacterActionType.SkillQ, 2);
SetAutoCooldown(CharacterActionType.SkillQ, false);

// 재장전이 끝난 뒤에만 쿨타임을 시작하는 예
StartCooldown(CharacterActionType.SkillQ);
```

`StartCooldown(action)`은 수치 파일 또는 `SetCooldownDuration(action, seconds)`으로 정한 기본 시간을 쓴다. `StartCooldown(action, seconds)`은 이번 한 번만 `seconds`로 시작한다. 즉, 기본 시간을 계속 바꾸려면 `SetCooldownDuration`을 쓰고, 한 번만 다른 시간을 쓰려면 두 번째 함수를 쓴다.

6.7 이동 잠금과 슬로우

이동 제한도 `CharacterBase`가 이미 구현했다. 캐릭터 하위 클래스는 스팀, 속박, 공격 효과가 필요할 때 아래 함수만 호출한다.

```
SetMovementEnabled(false);           // 일반 이동을 막는다.
ApplySlow(target, 0.4f, 2f);         // 대상 이동 속도를 40% 줄이고 2초 유지한다.
bool canMove = IsMovementEnabled();
float slow = SlowRatio;
bool slowed = IsSlowed;
```

`SetMovementEnabled(false)`는 수평 이동, 점프, 자동 점프, 새 대시를 막고 진행 중인 대시도 취소한다. 수평 속도는 즉시 0이 되지만 수직 속도는 유지되어 낙하와 중력은 계속 처리된다. `ApplySlow`는 더 강한 슬로우를 우선하고, 같은 강도면 시간을 새로 시작한다. `SlowRatio`는 현재 느려진 비율이고 `IsSlowed`는 실제 슬로우가 남아 있는지 알려 준다.

6.8 나중에 실행할 코드

`CharacterBase`가 이미 구현한 타이머를 쓴다. `ScheduleTimer`는 나중에 한 번 실행할 일을 등록하고, `CancelTimer`는 아직 실행 전인 일을 취소한다.

```
CharacterTimerHandle reload = ScheduleTimer(1.5f, () =>
{
    AddActionCharges(CharacterActionType.SkillQ, 1);
});

CancelTimer(reload); // 취소할 일이 없으면 false를 돌려준다.
```

6.9 바라보는 방향과 뒤 판정

방향 값도 `CharacterBase`가 이미 관리한다. 별도로 방향을 저장하거나 동기화하지 않는다.

```
if (IsBehindTarget(target, 120f) && IsFacingRight)
{
    // 대상 뒤쪽에 있을 때만 실행할 규칙
}

int direction = FacingDirection; // 오른쪽은 1, 왼쪽은 -1
bool right = IsFacingRight;
bool left = IsFacingLeft;
```

`IsBehindTarget(target, rearArcAngle)`의 각도는 대상의 뒤쪽 범위다. 예를 들어 `120f`는 뒤쪽 120도 안에 있는지 확인한다.

6.10 데미지와 투사체 이벤트

`DealDamage`, `SpawnProjectile`, `DespawnProjectile`의 실제 처리와 네트워크 처리는 `CharacterBase`가 이미 구현한다. 캐릭터 하위 클래스는 공격 규칙만 작성한다.

```
protected override float ModifyOutgoingDamage(
    CharacterBase target,
    float damage,
    CharacterDamageSource source)
{
    return IsBehindTarget(target, 120f) ? damage * 1.5f : damage;
}

protected override void OnProjectileDespawnd(
    CharacterProjectile projectile,
    ProjectileDespawnReason reason,
    CharacterBase hitTarget)
{
    if (reason == ProjectileDespawnReason.HitCharacter)
    {
        // hitTarget에 맞았을 때만 필요한 효과
    }
}
```

`ModifyOutgoingDamage`와 `OnProjectileDespawnd`는 필요한 캐릭터만 override한다. `OnDamageDealt`는 직접 공격과, 발사자 `CharacterBase`를 찾을 수 있어 원래 발사자에게 처리된 투사체 공격 모두에서 호출된다. 투사체의 발사자를 찾을 수 없으면 이 콜백은 호출되지 않는다.

`OnProjectileDespawnd`의 `reason`은 다음 중 하나다. `HitCharacter`는 캐릭터 명중, `HitWall`은 벽 명중, `LifetimeExpired`는 유지 시간 종료, `Manual`은 `DespawnProjectile`로 직접 끝낸 경우다. `hitTarget`은 `HitCharacter`일 때만 대상이고, 나머지는 `null`이다.

6.11 자주 읽는 현재 상태

값	뜻
CurrentHealth	현재 체력
MaxHealth	최대 체력
CurrentHealthPercent	현재 체력 비율
IsDead	사망 상태
IsLocalPlayer	이 컴퓨터가 조작하는 캐릭터인지
Cooldowns	행동별 쿨타임 조회 기능
Movement	지면, 대시, 이동 상태
AimDirection	현재 조준 방향
AimAngle	현재 조준 각도
AttackOrigin	근접 공격 시작 위치
ProjectileOrigin	투사체 시작 위치
EffectOrigin	시각 효과 시작 위치
WeaponRoot	무기를 붙이는 부모 위치

6.12 필요할 때만 구현하는 이벤트

함수	호출되는 시점
OnPassiveTick	지속 패시브 조건을 반복해서 확인할 때
OnDamaged	피해를 실제로 받았을 때
OnDamageDealt	직접 공격 또는 발사자가 확인된 투사체 공격이 대상에게 요청됐을 때
OnProjectileDespawnd	내 투사체가 명중, 벽, 시간 종료, 수동 종료로 사라졌을 때
OnDied	체력이 0이 되어 사망했을 때
OnJumped	점프가 시작됐을 때
OnLanded	공중에서 지면에 착지했을 때
OnSkillExecuted	어떤 행동 함수든 <code>true</code> 를 돌려 성공했을 때 한 번. 자동 쿨타임을 꺼 두었어도 호출되며, 쿨타임 시작은 나중에 미룰 수 있음
OnHealthChanged	체력이 바뀌었을 때
OnResetCharacter	라운드 초기화 등으로 캐릭터가 재설정될 때

7. 투사체 프리팹 만들고 캐릭터에 연결하기

총알, 화살, 마법탄을 사용하는 캐릭터는 `CharacterProjectile`이 붙은 투사체 프리팹을 만든다. 이 장에서는 필요한 컴포넌트, Inspector 설정, 캐릭터 스크립트 연결 방법만 설명한다.

7.1 투사체 프리팹 구조

```
ProjectileRoot
├─ NetworkObject
├─ CharacterProjectile
├─ Collider2D
└─ Visual
    └─ SpriteRenderer 또는 효과 오브젝트
```

필수 설정:

컴포넌트	설정
<code>NetworkObject</code>	네트워크 투사체 프리팹에 필수로 추가
<code>CharacterProjectile</code>	이동, 충돌, 데미지, 피격 효과 담당
<code>Collider2D</code>	투사체 크기에 맞게 설정. Trigger 사용 권장
<code>Visual</code>	스프라이트의 앞 방향을 확인하고 각도 보정값 설정

현재 프리팹에 필요한 네트워크 컴포넌트는 `NetworkObject`다. `NetworkTransform`은 추가하지 않는다.

7.2 Inspector 설정값

Inspector 이름	쉬운 설명	설정 예
Lifetime	아무것도 맞지 않았을 때 자동으로 사라지는 시간	3초에서 5초
Wall Layer	맞으면 사라질 벽과 지형 Layer	Ground, Wall
Collision Skin Width	충돌 누락을 줄이는 여유값. 특별한 이유가 없으면 기본값 유지	0.03
Projectile Angle Offset	투사체 그림이 이동 방향을 보도록 돌리는 보정각	+X 방향 그림은 0
Hit Vfx Prefab	충돌 위치에 생성할 시각 효과	불꽃, 먼지
Hit Vfx Angle Offset	충돌 효과 그림의 방향 보정각	+Y 방향 그림은 -90
Hit Vfx Surface Offset	효과가 벽 안에 묻히지 않도록 표면 밖으로 미는 거리	0.02

7.3 캐릭터 스크립트에서 사용하는 방법

```
[SerializeField] private CharacterProjectile projectilePrefab;
[SerializeField] private float projectileSpeed = 14f;
[SerializeField] private LayerMask targetLayer;

protected override bool OnBasicAttack(CharacterActionContext context)
{
    if (projectilePrefab == null)
        return false;

    SpawnProjectile(
        projectilePrefab,
        ProjectileOrigin.position,
        context.AimDirection,
        projectileSpeed,
        context.Damage,
        targetLayer);

    return true;
}
```

targetLayer는 캐릭터 스크립트에서 전달한다. **wallLayer**는 투사체 프리팹의 **CharacterProjectile** Inspector에서 설정한다.

7.4 투사체 확인 항목

- Projectile Origin에서 생성되는가?
- 스프라이트가 이동 방향을 바라보는가?
- 빠른 속도에서도 캐릭터와 벽을 통과하지 않는가?
- 발사한 캐릭터 자신을 맞이지 않는가?
- 캐릭터와 벽 Layer가 올바르게 나뉘어 있는가?
- 충돌 효과가 벽 안쪽에 묻히지 않는가?
- 수명이 끝나면 자동으로 제거되는가?
- 두 플레이어 화면에서 생성과 제거가 한 번씩만 보이는가?

8. 신규 캐릭터 개발 순서와 네트워크 주의점

8.1 권장 개발 순서

- 1 `CharacterTemplate.cs`를 복사해 캐릭터 스크립트를 만든다.
- 2 캐릭터 수치 파일을 만들고 체력, 이동, 데미지, 쿨타임을 입력한다.
- 3 공통 캐릭터 프리팹을 바탕으로 캐릭터별 프리팹을 만든다.
- 4 임시 `CharacterTemplate` 컴포넌트를 제거하고 캐릭터 스크립트를 추가한다.
- 5 `Body`, `Collider2D`, 공통 비주얼, 공격 위치 기준점을 연결한다.
- 6 평타 하나를 구현하고 판정과 쿨타임을 확인한다.
- 7 Q, E, 궁극기를 한 개씩 구현하고 매번 확인한다.
- 8 필요한 패시브 이벤트만 추가한다.
- 9 투사체와 시각 효과 프리팹을 연결한다.
- 10 자동 구성 검사와 Fusion 2인 테스트를 진행한다.

여러 스킬을 한꺼번에 구현하지 않는다. 평타부터 하나씩 확인하면 중복 실행과 잘못된 판정의 원인을 찾기 쉽다.

8.2 자동 구성 검사

캐릭터별 프리팹을 Project 창에서 선택한 뒤 다음 메뉴를 실행한다.

Tools > Project MS > Character > Validate Selected Character

검사하는 항목:

- `CharacterBase`를 상속한 캐릭터 컴포넌트
- `NetworkObject`
- `Rigidbody2D`
- `Collider2D`
- `CharacterVisualController`
- 캐릭터 수치 파일 연결
- 공통 움직임 설정 연결

8.3 캐릭터 스크립트에서 사용하지 않는 네트워크 코드

신규 캐릭터 스크립트에서는 다음 작업을 직접 구현하지 않는다. 이것들은 `CharacterBase`가 이미 처리한다.

- `[Networked]` Fusion 상태 변수 추가
- `[Rpc]` 또는 RPC 네트워크 함수 추가
- `Runner.Spawn`으로 네트워크 오브젝트 직접 생성
- 네트워크 권한 직접 검사
- 원격 캐릭터를 위한 별도 입력 처리

- **Rigidbody2D**를 이용한 별도 화면 전용 이동

대신 다음 공통 함수를 사용한다.

```
SpawnProjectile
FindEnemiesInCircle / Box / Line / Arc
DealDamage
Heal
StartDash
PlayActionEffect
```

투사체는 **CharacterBase.SpawnProjectile**으로 만들고, 종료가 필요하다면 **DespawnProjectile**을 사용한다. 캐릭터 스크립트에서 **CharacterProjectile.Initialize**을 호출하거나 **Runner.Spawn**을 쓰지 않는다.

공통 함수만으로 만들 수 없는 설치물, 소환수, 변신, 지속 장판, 충전 스킬은 캐릭터 스크립트에 Fusion 코드를 바로 추가하지 말고 공통 시스템 담당자에게 필요한 동작을 요청한다.

8.4 Fusion 2인 테스트

- 1 Host와 Client를 실행한다.
- 2 각 플레이어가 자기 캐릭터만 조작하는지 확인한다.
- 3 양쪽 화면에서 이동과 조준 방향이 같은지 확인한다.
- 4 평타와 스킬이 한 번 입력에 한 번만 실행되는지 확인한다.
- 5 투사체가 양쪽에서 같은 대상과 벽에 충돌하는지 확인한다.
- 6 체력과 사망 상태가 양쪽에서 같은지 확인한다.
- 7 라운드 재시작 후 체력, 위치, 쿨타임이 초기화되는지 확인한다.

9. 마지막 점검 목록

프리팸과 수치

- 캐릭터 수치 파일이 연결되어 있는가?
- `NetworkObject`, `NetworkRigidbody2D`, `Rigidbody2D`, `Collider2D`가 있는가?
- `CharacterTemplate`와 캐릭터 스크립트가 동시에 붙어 있지 않은가?
- `Body`와 `CharacterVisualController`가 연결되어 있는가?
- 공격 위치 기준점이 실제 공격 위치와 맞는가?
- 사용하지 않는 위치 기준점을 억지로 만들지 않았는가?

평타, 스킬, 패시브

- 성공한 행동은 `true`, 취소한 행동은 `false`를 반환하는가?
- 데미지는 `DealDamage`를 사용하는가?
- 범위 검색은 `FindEnemies...`를 사용하는가?
- 투사체는 `SpawnProjectile`을 사용하는가?
- 기본 대시를 캐릭터마다 다시 구현하지 않았는가?
- 패시브는 상황에 맞는 이벤트 함수에 작성했는가?

투사체

- 투사체 프리팸에 `NetworkObject`, `CharacterProjectile`, `Collider2D`가 있는가?
- Target Layer와 Wall Layer가 구분되어 있는가?
- 빠른 속도에서도 벽과 캐릭터를 통과하지 않는가?
- 각도 보정값 때문에 그림이 뒤집히거나 옆을 보지 않는가?
- 피격 효과 위치와 방향이 자연스러운가?

네트워크

- 캐릭터 스크립트에 Fusion 네트워크 코드를 직접 추가하지 않았는가?
- 한 번 입력에 공격과 효과가 한 번만 생성되는가?
- 두 플레이어 화면에서 데미지와 사망 결과가 같은가?
- 라운드 초기화 후 상태가 정상으로 돌아오는가?

화면 표현

- 원격 캐릭터가 로컬 마우스를 따라가지 않는가?
- 좌우 반전 후 무기와 장식 방향이 올바른가?
- 점프, 착지, 피격 표현이 공통 비주얼 설정을 따르는가?
- 캐릭터별 시각 효과가 공통 프리팸에 잘못 저장되지 않았는가?